

Cartographer-SLAM Mapping

Cartographer-SLAM Mapping

1. Course Content
2. Cartographer Introduction
3. Start the Agent
4. Run the Example
 - 4.1 Start the Mapping Node
 - 4.2 Save the Map
 - 4.3 Save Pose Map in pbstream Format
5. Node Analysis
 - 5.1 View Node Communication Graph
 - 5.2 TF Transformation

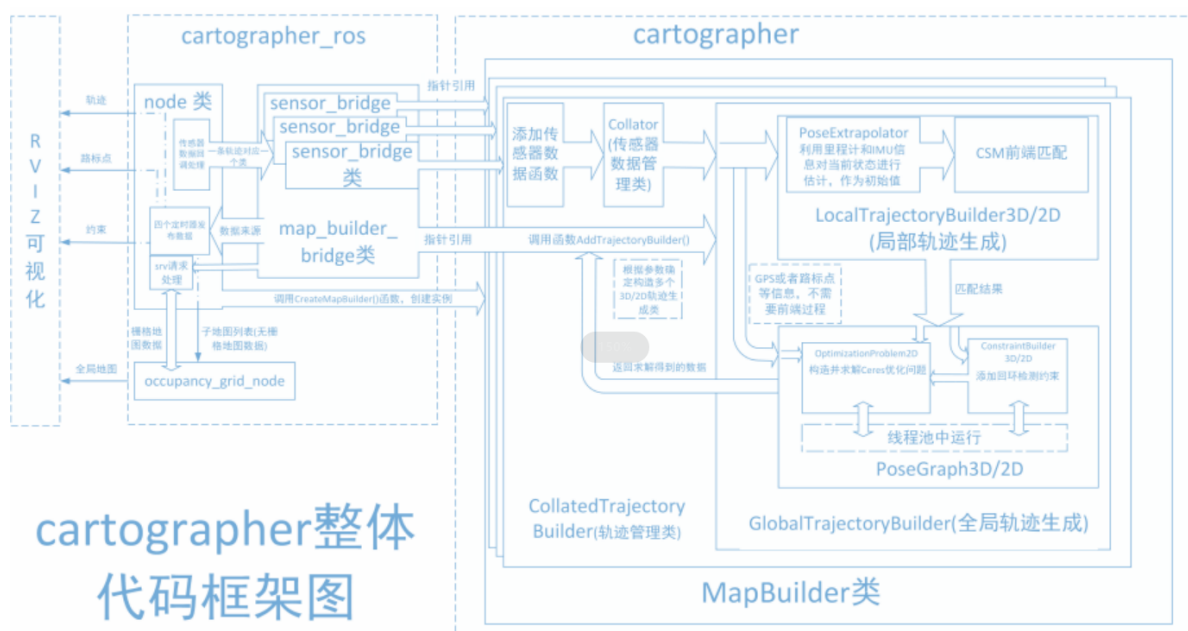
1. Course Content

1. Run the sample Cartographer SLAM algorithm to perform SLAM mapping and save the grid map and pose map.

2. Cartographer Introduction

Cartographer is a 2D and 3D SLAM library open-sourced by Google, supported by the ROS system. It is a graph-based SLAM algorithm (with multi-threaded backend optimization and problem optimization built with Ceres). It can combine data from multiple sensors (e.g., LIDAR, IMU, and cameras) to simultaneously compute the sensor's position and map the surrounding environment.

The source code of Cartographer mainly consists of three parts: `cartographer`, `cartographer_ros`, and `ceres-solver` (for backend optimization).



Cartographer uses a mainstream SLAM framework, which is a three-stage process of feature extraction, loop closure detection, and backend optimization. A certain number of `LaserScan`s form a `submap`, and a series of submaps constitute the global map. The cumulative error is small when building a submap from `LaserScan`s over a short period, but a large cumulative error can occur when building a global map from submaps over a long period. Therefore, loop closure detection is needed to correct the positions of these submaps. The basic unit for loop closure detection is the submap, and the `scan_match` strategy is used for this. The key aspects of Cartographer are the creation of submaps by fusing multi-sensor data (odometry, IMU, LaserScan, etc.) and the implementation of the `scan_match` strategy for loop closure detection.

- `cartographer_ros`

`cartographer_ros` runs under ROS and can receive various sensor data as ROS messages. After processing, it publishes the results as messages, which is convenient for debugging and visualization.

3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
mircoROS_Agent
[1765183764.822476] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1765183764.826313] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x006(2), participant_
id: 0x000(1)
[1765183764.829482] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1765183764.834154] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1765183764.838233] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x007(2), participant_
id: 0x000(1)
[1765183764.842504] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1765183764.847468] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

4. Run the Example

4.1 Start the Mapping Node

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

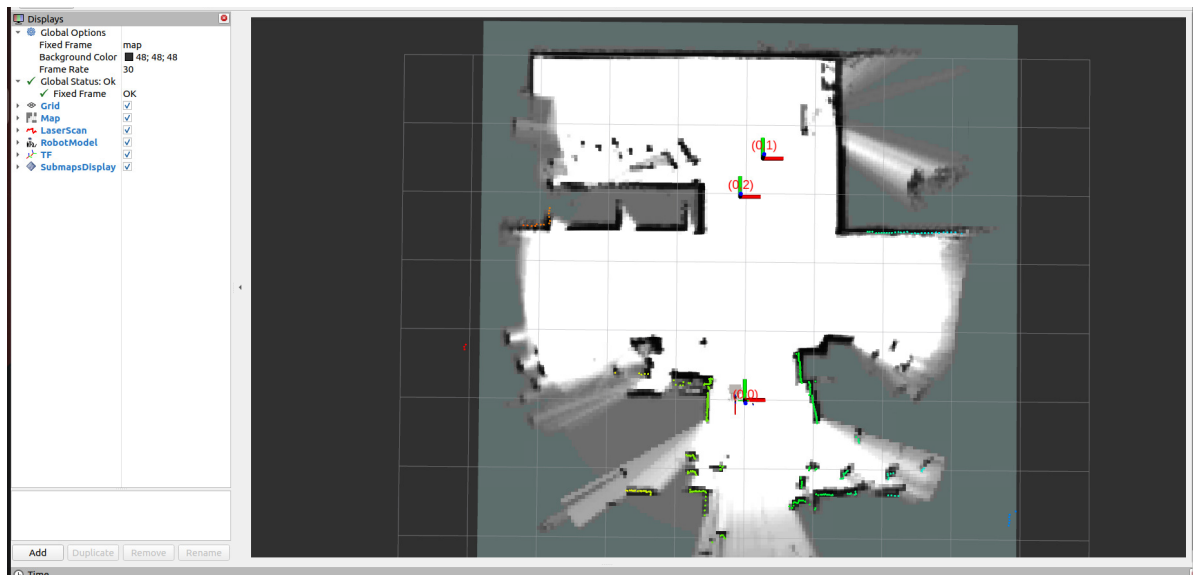
```
exec_m3
```

(ORIN board)

```
ros2 launch m3_bringup cartographer.launch.py
```

(RDK & PI5 boards) Run in separate steps, with visualization launched on the accompanying virtual machine Rosmaster-M3-Ubuntu22.04 ,

```
# On the virtual machine
rviz2 -d
/home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/slam_cartographer_rviz.rviz
iz
# On the host machine
ros2 launch m3_bringup cartographer.launch.py gui:=False
```



- To control the vehicle's movement, you can use either keyboard or joystick control. Here, we use keyboard control as an example:

```
keyboard_control
```

- Click on the terminal window with the mouse and press **z** to reduce the speed appropriately. Press **I**, **<**, **J**, **L** to control the car to move forward, backward, turn left, and turn right, respectively, to complete the mapping slowly.

```
j k l
m , .

For Holonomic mode (strafing), hold down the shift key:
-----
U I O
J K L
M < >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%
```

4.2 Save the Map

Open a new terminal on the vehicle and save the map.

```
ros2 launch m3_bringup save_map.launch.py
```

[!TIP]

Optional Parameters:

`map_name`: Specifies the name of the saved map, with the default value being `map`.

For example, to save a map named `test.yaml`, add the parameter: `ros2 launch m3_bringup save_map.launch.py map_name:=test`

The terminal prompt **Map saved successfully** indicates that the map was saved successfully.

```
jetson@yahboom: ~
jetson@yahboom: ~ 98x24
-yahboom-1015116
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [map_saver_cli-1]: process started with pid [1015173]
[map_saver_cli-1] [INFO] [1765187109.641023796] [map_saver]:
[map_saver_cli-1] map_saver lifecycle node launched.
[map_saver_cli-1] Waiting on external lifecycle transitions to activate
[map_saver_cli-1] See https://design.ros2.org/articles/node_lifecycle.html for more informat
ion.
[map_saver_cli-1] [INFO] [1765187109.641307905] [map_saver]: Creating
[map_saver_cli-1] [INFO] [1765187109.642470006] [map_saver]: Configuring
[map_saver_cli-1] [INFO] [1765187109.648158841] [map_saver]: Saving map from 'map' topic to '/home
/jetson/map/map' file
[map_saver_cli-1] [WARN] [1765187109.676050702] [map_io]: Image format unspecified. Setting it to:
pgm
[map_saver_cli-1] [INFO] [1765187109.677272806] [map_io]: Received a 157 X 179 map @ 0.05 m/pix
[map_saver_cli-1] [INFO] [1765187109.702157746] [map_io]: Writing map occupancy data to /home/jets
on/map/map.pgm
[map_saver_cli-1] [INFO] [1765187109.707867254] [map_io]: Writing map metadata to /home/jetson/map
/map.yaml
[map_saver_cli-1] [INFO] [1765187109.708400751] [map_io]: Map saved
[map_saver_cli-1] [INFO] [1765187109.708436304] [map_saver]: Map saved successfully
[map_saver_cli-1] [INFO] [1765187109.717480396] [map_saver]: Destroying
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 1015173]
jetson@yahboom:~$
```

- The map save path is as follows:

```
~/map/map.yaml
```

- This includes a PGM image and a YAML file, `map.yaml`.

```
image: map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Parameter analysis:

- `image`: The path to the map file, which can be an absolute or relative path.
- `mode`: This attribute can be `trinary`, `scale`, or `raw`, depending on the chosen mode. `trinary` is the default.
- `resolution`: The resolution of the map, in meters/pixel.
- `origin`: The 2D pose (x, y, yaw) of the bottom-left corner of the map. Here, `yaw` is a counter-clockwise rotation (yaw=0 means no rotation). Many parts of the system currently ignore the yaw value.
- `negate`: Whether to invert the meaning of white/black and free/occupied (the interpretation of thresholds is not affected).
- `occupied_thresh`: Pixels with an occupancy probability greater than this threshold are considered fully occupied.
- `free_thresh`: Pixels with an occupancy probability less than this threshold are considered fully free.

4.3 Save Pose Map in pbstream Format

The `pbstream` format map file is used for Cartographer's localization and navigation, which will be explained in subsequent chapters.

- Open a terminal and enter the following command:

```
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename:
'/home/jetson/map/map.pbstream'}"
```

[!WARNING]

`filename`: `'/home/jetson/yahboom_map.pbstream'` is the path to save the file. When saving, replace it with the corresponding path in the `~/map/` directory on your board.

5. Node Analysis

5.1 View Node Communication Graph

- Run the following command in the terminal:

```
ros2 run rqt_graph rqt_graph
```

